EPOCH

INDIVIDUAL WORK DIVISION & DELIVERY PLAN

PERSON 4

Pranav Aditya

Frontend Workflow, Evidence Presentation & User Documentation Owner

Allocation	Project	Architecture baseline
15% of total team effort	Project Axiom — EPOCH: Universal Intelligence & Discovery Engine	EPOCH_Final_System_Architecture_v2.md

Purpose: This document assigns Person 4's individual responsibilities, deliverables, boundaries and acceptance criteria. It describes planned work; it does not claim the work is already implemented.

Core principle: Build a small, reliable interface through which a user can complete and inspect one investigation. The engine and backend own scientific logic, permissions and verdicts; the frontend presents those results faithfully.

Review milestone: Axiom Review One — 29 November 2026. Sprint dates and intermediate checkpoints should be agreed with the team.

1. Role, ownership and boundaries

Pranav owns the EPOCH user interface and the documentation needed for another teammate or reviewer to run the Core demonstration. The 15% allocation is intentionally focused: frontend workflow, evidence presentation, API integration and user-facing documentation.

Responsibilities

- **Frontend foundation:** Build the React + TypeScript + Vite application shell, navigation, reusable layout, form controls, tables, status indicators and error states.
- **Investigation workflow:** Create the minimum screens for investigation overview, dataset/version, exploration, hypotheses, analysis plan, approval, run status, verdict and report.
- **Evidence presentation:** Display evidence items, claims, supporting/contradicting links, run history and provenance references returned by the API.
- **API integration:** Maintain typed request/response interfaces that match the agreed backend contract; handle loading, empty, validation, permission and server-error states.
- **Testing:** Add component and key workflow tests, including common API failures and invalid form input.
- **Documentation/demo:** Write setup instructions, a short user workflow guide and a repeatable Review One demonstration script.

Out of scope — do not take ownership of these

- Statistical tests, p-values, confidence intervals, α/K correction, scientific verdict calculations or acceptance logic.
- Row-hash splitting, holdout sealing, lineage budgets, frozen-plan enforcement, audit-chain signing or sandbox execution.
- Executing uploaded code, formulas, notebooks or arbitrary commands in the browser.
- Optional AI hypothesis generation/narration, advanced graph visualisation, collaboration extras or other extensions before Core is complete.
- Inventing API fields or changing the meaning of backend statuses. Raise contract gaps with the technical lead/backend owner.

Definition of done

A person unfamiliar with the codebase can follow the setup guide, launch the app and complete or inspect the agreed end-to-end demonstration. The UI accurately reflects server-owned states, handles common failures and never presents a draft, exploratory or post-hoc result as confirmatory.

2. Deliverables and acceptance criteria

Deliverable	Expected output	Acceptance evidence
Frontend shell	React + TypeScript + Vite app, navigation, reusable components and environment configuration.	Clean install/build; no secrets committed; basic responsive layout.
Core workflow screens	Dataset, explore, hypothesis, plan, approval, execution and report views.	Reviewer can navigate the Core workflow without developer-only steps.
Typed API client	Central API client and TypeScript types aligned with backend schemas.	Requests match documented endpoints; loading and errors are handled.
Evidence & run ledger	Evidence/claim list, run history, statuses, provenance and report details.	Displayed records link to API identifiers; no fabricated metrics.
Tests	Component and workflow tests for critical screens and failure states.	Key user path and API error handling covered; tests documented.
User documentation	README/setup, workflow guide and demo script.	Another teammate can start the app and reproduce the demo.

UI behaviour rules

- Show workflow states exactly as supplied by the API: draft, awaiting approval, frozen, running, completed, failed, exploratory, confirmatory or post-hoc, as applicable.
- Display verdict wording and numerical results exactly as returned by the backend. Never calculate or upgrade a verdict in the browser.
- Show provenance identifiers and timestamps where available; make missing information explicit.
- Use server authorization as the source of truth. Hiding/disabling a button is not a substitute for backend permission checks.

- Do not add a client-side route or query that attempts to retrieve sealed confirmation-split rows.

3. Screen-level scope

Screen	Minimum content	Constraints
Investigation home	Title, question, current stage, dataset version, latest activity and next allowed action.	No AI-generated summary required for Core.
Dataset & Explore	Dataset metadata/profile, quality warnings, saved observations and permitted exploration context.	Never expose confirmation-split rows.
Hypotheses	Hypothesis statement, rationale, evidence links, competing explanation and revision history.	Edits remain drafts until saved; status changes are server-controlled.
Analysis plan	Primary outcome, selected test, declared covariates/variants and assumptions.	Show frozen/locked state; do not calculate statistics.
Approval & run	Author/approver, approval state, run status, progress/error and run identifier.	Trigger only the approved backend action; no direct execution.
Verdict & report	Backend verdict, estimate/interval, limitations, evidence and provenance.	Do not overstate the conclusion.
History	Dataset versions, hypothesis revisions, approvals and runs in chronological order.	Preserve prior records; later runs must not rewrite earlier results.

Suggested frontend structure

Use a modular structure and align it with the repository conventions before creating duplicate abstractions:

src/api/ — API client and shared types
src/features/investigations/ — investigation overview
src/features/datasets/ — dataset and exploration views
src/features/hypotheses/ — hypothesis editor
src/features/plans/ — plan and approval views
src/features/runs/ — execution status and run ledger
src/features/reports/ — verdict/report presentation
src/components/ — reusable UI components
src/routes/ — route definitions.

Accessibility and usability minimum

- Provide labels for form fields, keyboard-accessible controls and visible focus states.
- Use readable contrast and status text; never communicate status by colour alone.
- Place validation messages beside the relevant field and preserve entered values after recoverable errors.
- Provide useful empty states and actionable error messages; do not expose stack traces, secrets or internal infrastructure details.

4. Dependencies and team coordination

Dependency / owner	What Person 4 needs	Coordination action
Adithya — Technical Lead / Integrity Engine	Agreed domain terms, workflow states, verdict vocabulary and stable engine-facing concepts.	Confirm UI labels and report fields; raise any ambiguity before implementing assumptions.
Sushma Gouda — Backend Platform / API	Endpoint list, request/response schemas, auth rules, error format and sample responses.	Agree API contract early; use mock data only until endpoints are available, then replace it.
Atul Dhull — Execution / Sandbox / Verification	Job lifecycle, run status vocabulary, progress/error fields and artifact access rules.	Integrate run-status and artifact views only through approved backend interfaces.
Pranav Aditya — Frontend / Docs	UI, typed API integration, frontend tests and user/demo documentation.	Own the frontend checklist and report blockers at each team sync.

Working agreement

- Create a small frontend task branch and submit focused pull requests; do not commit directly to the shared main branch unless the team agrees.
- Keep API types in one place. When a contract changes, update types and tests together and notify the backend owner.
- Use clearly labelled mock fixtures for UI work before the API is ready; never present mock results as real experiment output.
- Raise blocked dependencies early rather than building speculative screens around unconfirmed fields.

- At handoff, include run instructions, environment variable names (not secret values), test commands and known limitations.

5. Milestones and weekly evidence

Stage	Target output	Evidence to show
Start / setup	Repository access, frontend scaffold, run/build instructions and agreed API contract.	Successful local start; initial screen; documented assumptions.
Workflow slice	Dataset → explore → hypothesis → plan → approval screens connected or using labelled fixtures.	Screenshots or short walkthrough; form and navigation tests.
Execution & report	Run status, evidence/claim list, verdict/report and history views integrated with backend.	End-to-end API walkthrough; visible provenance and error states.
Hardening	Accessibility/usability pass, tests, README and demo script.	Build/test output; setup verified by another teammate.
Review One — 29 Nov 2026	Stable, rehearsed demonstration and clear explanation of Person 4 contribution.	Live walkthrough, contribution summary and known limitations.

Weekly update format

At each team check-in, report: (1) completed items with links/commits/screenshots, (2) next tasks, (3) blockers and the person needed to resolve them, (4) API contract changes, and (5) any scope risks. Keep the update factual and short.

6. Review One contribution and final handoff

What to demonstrate

- Launch the frontend from the documented setup steps.
- Open an investigation and navigate through dataset, hypothesis and analysis-plan information.
- Show approval and run status as returned by the backend.
- Open the report and trace a displayed claim to its evidence/run identifier.
- Demonstrate one useful validation or failure state and explain how the UI avoids misrepresenting results.

Handoff checklist

- Frontend code committed and pushed to the agreed branch; pull request reviewed or ready for review.
- README includes prerequisites, install/start/build/test commands and required environment variable names.
- API integration types and endpoint assumptions are documented.
- Critical workflow tests pass; known gaps and mock fixtures are clearly labelled.
- Demo script and screenshots are available to the team.
- No credentials, private datasets or unapproved user data are committed.

Success measures

The contribution is complete when the Core user journey is understandable, the UI is integrated with the agreed API contract, evidence and results are traceable, common failures are handled, and a teammate can reproduce the demonstration from the documentation.

Scope reminder: 15% means a focused frontend and documentation workstream. Prioritise a working, trustworthy Core workflow over optional polish or advanced visualisations.

Prepared from the EPOCH Final System Architecture v2 baseline. The architecture is a proposed design; this work plan is an assignment, not evidence of implementation or security certification.